# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO1 – Imitation (L1) | **Assessment:** | Assessment Tool 1 – Application Based Questions |
| **Weightage:** | 40% of CIA | | |
| **CO1 Statement:** | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) | | |
| **Student Name:** | P. Nandu Vardhan | **Reg. No.:** | 192425067 |
| **Section / Batch:** | B.Tech AIML-2024 | **Date:** | 10-10-26 |

**Code of Conduct**

*I P. Nandu Vardhan(192425067) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

**Instructions**

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

| Section / Topic | Focus | Q Nos |
|---|---|---|
| **Information Extraction** | Regex-Based Information Extraction | 1 |
| **Pattern Matching** | Regex Pattern Matching | 2 |
| **Regular Expression Patterns** | Regex-Based Input Validation | 3 |

**Section 1: Regular Expressions – Resume Information Extraction**

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

**Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:**

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

# Section 1: Regular Expressions – Resume Information Extraction

## 1. Aim

To design a Python-based resume information extraction system using Regular Expressions (Regex) to extract important candidate details from resume text, generate a structured profile summary, and identify candidates who meet the minimum eligibility criteria.

## 2. Implementation Approach

The system takes resume text as input and uses Python's `re` module to identify and extract relevant information. The extracted details are organized into a structured format for easy analysis and candidate shortlisting.

Step 1: Input Resume Text

Collect resume content in text format. For a simple implementation, the resume text can be stored directly in a Python string.

Step 2: Extract Candidate Name

Use a Regular Expression to identify a candidate's name, assuming it appears near the beginning of the resume or follows a label such as `Name:`.

Step 3: Identify Email Addresses

Use a pattern that matches the standard email structure, including a username, the `@` symbol, a domain name, and an extension. The `re.findall()` function can identify email addresses in the resume.

Step 4: Extract Mobile Numbers

Use a Regex pattern to identify mobile numbers, considering common formats such as numbers with or without country codes, spaces, and hyphens.

Step 5: Detect Technical Skills

Maintain a list of required technical skills, such as Python, Java, SQL, Machine Learning, and NLP. Search for these skills in the resume text using case-insensitive matching.

Step 6: Extract Years of Experience

Use Regex patterns to identify phrases such as `2 years of experience` or `3.5 years experience`. Convert the extracted value into a numeric format for eligibility checking.

Step 7: Generate a Structured Profile Summary

Store the extracted information in a Python dictionary containing the candidate's name, email, mobile number, technical skills, and years of experience. Display the dictionary as a readable candidate profile.

Step 8: Check Minimum Eligibility

Apply the following conditions to determine whether a candidate qualifies for shortlisting:

- Minimum experience: 2 years.
- Required skill: Python.

A candidate is marked Eligible when both conditions are satisfied; otherwise, the candidate is marked Not Eligible.

# 3. Key Part of the Algorithm

The main algorithm follows these steps:

1. Read the resume text.
2. Apply Regex patterns to extract the candidate's name, email, and mobile number.
3. Search for the specified technical skills and store the matching skills in a list.
4. Extract years of experience and convert the value into a number.
5. Create a structured profile using the extracted information.
6. Check whether experience is at least 2 years and Python is present in the detected skills.
7. Display the candidate's profile and eligibility status.

# 4. Expected Output

The system generates a structured summary similar to the following:

| Field | Extracted Information |
|---|---|
| Candidate Name | Arun Kumar |
| Email | arun.kumar@example.com |
| Mobile Number | 9876543210 |

| | |
|---|---|
| Technical Skills | Python, SQL, Machine Learning |
| Years of Experience | 3 years |
| Eligibility Status | Eligible |

Note: The candidate information shown above is illustrative.

# 5. Conclusion

The Regular Expressions-based resume information extraction system automates the identification of essential candidate details from text-based resumes. By using Python's `re` module, lists, dictionaries, and simple conditional statements, the system reduces manual screening effort and helps HR teams shortlist candidates based on experience and technical skills. This approach is suitable for an intermediate-level Python implementation. Regex extraction depends on the formatting of the resume, so the results may require validation for inconsistent or unusual formats.

### Implementation

```
# 3. Course Code: Example CSE101
results["Course Code"] = bool(
    re.fullmatch(r"[A-Za-z]{2,4}\d{3}", course_code)
)

# 4. Semester: Must be between 1 and 8
results["Semester"] = bool(
    re.fullmatch(r"[1-8]", semester)
)

# 5. Mobile Number: Indian 10-digit mobile number
results["Mobile Number"] = bool(
    re.fullmatch(r"[6-9]\d{9}", mobile)
)

# Display individual validation results
print("\n--- Validation Report ---")

for field, valid in results.items():
    if valid:
        print(field, ": Valid")
    else:
        print(field, ": Invalid")

# Final registration status
print("\n--- Final Registration Status ---")

if all(results.values()):
    print("Registration Successful")
else:
    print("Registration Failed")
    print("Please correct the invalid fields.")
```

```
Enter Register Number:  22CSE001
Enter Institutional Email:  student@university.edu.in
Enter Course Code:  CSE101
Enter Semester (1-8):  4
Enter Mobile Number:  9876543210

--- Validation Report ---
Register Number : Valid
Institutional Email : Valid
Course Code : Valid
Semester : Valid
Mobile Number : Valid

--- Final Registration Status ---
Registration Successful
```

**Section 2: Regular Expressions – Product Search System**

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

**Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:**

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

# Section 2: Regular Expressions – Product Search System

## 1. Aim

To design a Python-based product search system using Regular Expressions (Regex) that allows users to search for products using exact keywords, prefixes, suffixes, and partial keywords. The system also supports case-insensitive searches and generates a report showing the number of matching products.

## 2. Implementation Approach

The system maintains a list of product names and uses Python's `re` module to search for matching products based on different search methods.

Step 1: Create a Product List

Store product names in a Python list. This list acts as the product database for the search system.

Step 2: Exact Keyword Search

Use a Regex pattern with word boundaries to find products containing an exact keyword. This helps distinguish a complete word from a word that merely contains the same letters.

Step 3: Prefix-Based Search

Use a pattern with the `^` symbol to identify product names that begin with the specified keyword.

Step 4: Suffix-Based Search

Use the `$` symbol to match product names that end with the specified keyword.

Step 5: Partial Keyword Search

Search for a keyword anywhere within a product name. This allows users to find products even when they enter only a part of the product name.

Step 6: Case-Insensitive Search

Use the `re.IGNORECASE` flag so that searches return matches regardless of whether the user enters uppercase or lowercase letters.

Step 7: Display Matching Products

Store all matching product names in a list and display them to the user. If no products match the search keyword, display an appropriate message.

Step 8: Generate a Search Report

Count the matching products using Python's `len()` function. Display the search method, keyword, matched product names, and total number of results.

# 3. Key Part of the Algorithm

The main algorithm follows these steps:

1. Initialize a list containing product names.
2. Accept a keyword and the search method from the user.
3. Construct the appropriate Regex pattern based on the selected method.
4. Apply the pattern to each product name using Python's `re` module.
5. Enable case-insensitive matching using `re.IGNORECASE`.
6. Store all matching product names in a separate list.
7. Display the matching products and calculate their total count.
8. Generate a report containing the keyword, search type, and number of matching products.

# 4. Expected Output

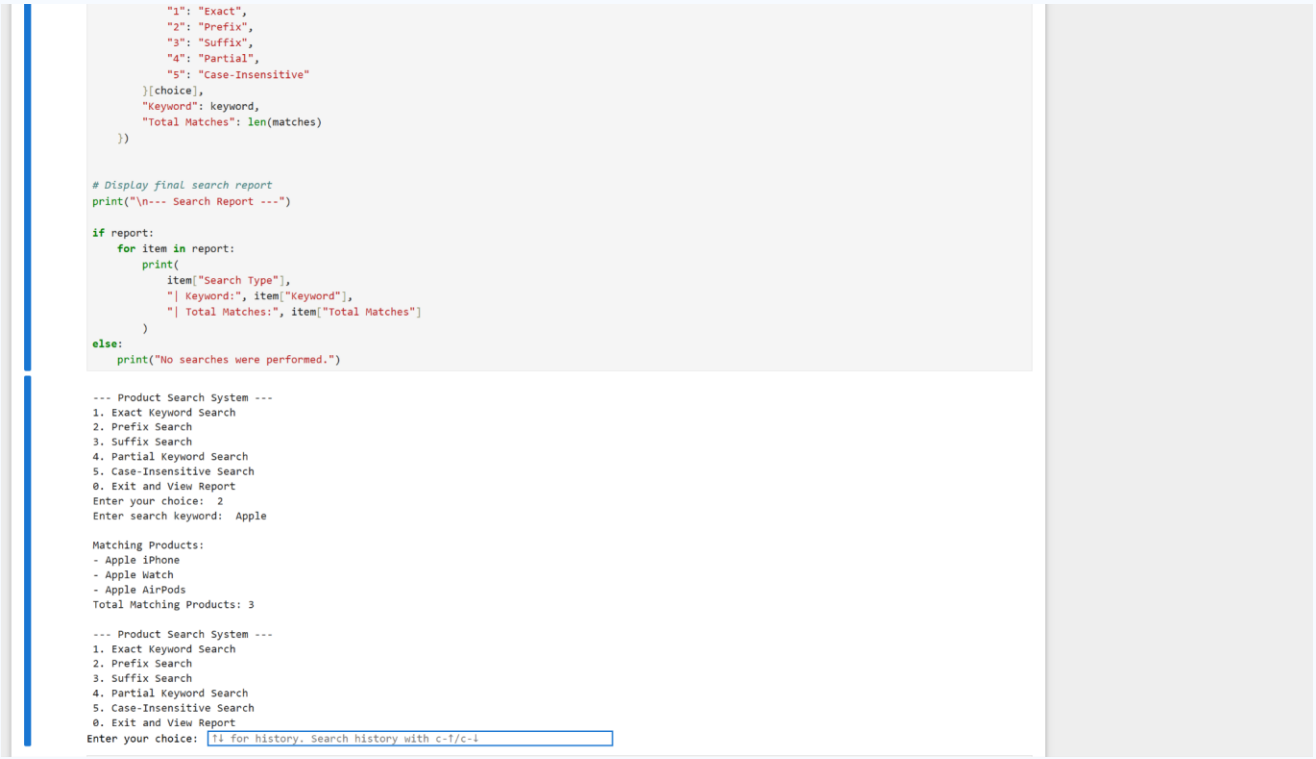
For example, consider a product list containing Apple iPhone, Apple Watch, Samsung Phone, Wireless Headphones, and Phone Case.

| Search Type | Keyword | Matching Products | Total Matches |
|---|---|---|---|
| Exact keyword | Phone | Samsung Phone, Phone Case | 2 |
| Prefix-based | Apple | Apple iPhone, Apple Watch | 2 |
| Suffix-based | Phone | Samsung Phone | 1 |
| Partial keyword | Head | Wireless Headphones | 1 |
| Case-insensitive | apple | Apple iPhone, Apple Watch | 2 |

Note: These results are illustrative. Exact keyword matching is assumed to match a complete word anywhere in the product name.

# 5. Conclusion

The Regular Expressions-based product search system improves product discovery by supporting multiple keyword-matching methods. Using Python's `re` module, lists, loops, and conditional statements, the system can identify matching products, handle different letter cases, and generate search reports. This provides a simple intermediate-level approach to implementing flexible product search functionality for an e-commerce application.



```
            "1": "Exact",
            "2": "Prefix",
            "3": "Suffix",
            "4": "Partial",
            "5": "Case-Insensitive"
        }[choice],
        "Keyword": keyword,
        "Total Matches": len(matches)
    })


# Display final search report
print("\n--- Search Report ---")

if report:
    for item in report:
        print(
            item["Search Type"],
            "| Keyword:", item["Keyword"],
            "| Total Matches:", item["Total Matches"]
        )
else:
    print("No searches were performed.")
```

```
--- Product Search System ---
1. Exact Keyword Search
2. Prefix Search
3. Suffix Search
4. Partial Keyword Search
5. Case-Insensitive Search
0. Exit and View Report
Enter your choice:  2
Enter search keyword:  Apple

Matching Products:
- Apple iPhone
- Apple Watch
- Apple AirPods
Total Matching Products: 3

--- Product Search System ---
1. Exact Keyword Search
2. Prefix Search
3. Suffix Search
4. Partial Keyword Search
5. Case-Insensitive Search
0. Exit and View Report
Enter your choice:  ↑↓ for history. Search history with c-↑/c-↓
```

## Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

**Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:**

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

# Section 3: Regular Expressions – University Registration System

## 1. Aim

To design a Python-based student registration validation system using Regular Expressions (Regex) to verify student details before course enrollment, display appropriate validation messages, and generate a final registration status report.

## 2. Implementation Approach

The system accepts student registration details as input and uses Python's `re` module to check whether each field follows the required format.

Step 1: Validate Student Register Number

Define a Regex pattern based on the university's register number format. For example, a format such as `22CSE001` can be checked for two digits, three letters, and three digits.

Step 2: Validate Institutional Email Address

Check whether the email follows a valid email structure and uses the university's approved domain, such as `student@university.edu.in`. The domain should be adjusted according to the actual institution.

Step 3: Validate Course Code

Use a Regex pattern to verify that the course code follows the expected format, such as `CSE101` or `CS102`, with letters followed by digits.

Step 4: Validate Semester Information

Check whether the entered semester is within the permitted range. For example, an undergraduate program may allow semester values from 1 to 8.

Step 5: Validate Student Mobile Number

Use a Regex pattern to check whether the mobile number contains exactly 10 digits. For an Indian mobile number, the pattern can also require the first digit to be between 6 and 9.

Step 6: Display Validation Messages

Validate each field separately and display a suitable message, such as Valid register number or Invalid mobile number. This helps students identify and correct errors.

Step 7: Generate the Final Registration Report

Store the validation results for all fields. If every field is valid, display Registration Successful. Otherwise, display Registration Failed and indicate which fields need correction.

## 3. Key Part of the Algorithm

The main algorithm follows these steps:

1. Accept the student's register number, institutional email, course code, semester, and mobile number.
2. Define Regex patterns according to the university's validation rules.
3. Use Python's `re.fullmatch()` function to check each field against its complete expected pattern.
4. Validate the semester against the permitted range.
5. Display individual success or error messages for every field.
6. Store the validation results using Boolean variables.
7. Check whether all validation results are `True`.
8. Display the final registration status based on the result.

# 4. Expected Output

The following example illustrates how the system may display the results:

| Field | Sample Input | Validation Result |
|---|---|---|
| Register Number | 22CSE001 | Valid register number |
| Institutional Email | student@university.edu.in | Valid institutional email |
| Course Code | CSE101 | Valid course code |
| Semester | 4 | Valid semester |
| Mobile Number | 9876543210 | Valid mobile number |

Final Registration Status: Registration Successful

If a student enters an invalid email address or a semester outside the permitted range, the system displays the corresponding error message and marks the registration as unsuccessful.

Note: The register number, email domain, course code, and semester formats are illustrative. They should be adjusted to match the university's actual requirements.

# 5. Conclusion

The Regular Expressions-based university registration system helps verify student information before course enrollment. Using Python's `re` module, pattern matching, conditional statements, and Boolean variables, the system validates individual fields, displays appropriate messages, and determines the final registration status. This provides a simple, intermediate-level approach to reducing errors in online student registration.

## Implementation

```
print("\n===== FINAL REGISTRATION REPORT =====")

if all(results.values()):
    print("Registration Status: SUCCESSFUL")
    print("All student details are valid.")
else:
    print("Registration Status: FAILED")
    print("Please correct the invalid fields.")

print("\nInvalid fields:")
invalid_fields = [
    field.replace("_", " ").title()
    for field, valid in results.items()
    if not valid
]

if invalid_fields:
    for field in invalid_fields:
        print("-", field)
else:
    print("None")
```

```
===== UNIVERSITY REGISTRATION VALIDATION =====

Register Number: 23CSE1045 - Valid
Email: 23cse1045@university.edu - Valid
Course Code: CSE101 - Valid
Semester: Semester 3 - Valid
Mobile: 9876543210 - Valid

===== FINAL REGISTRATION REPORT =====
Registration Status: SUCCESSFUL
All student details are valid.

Invalid fields:
None
```

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Understanding of Problem Context** | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| **Application of Concepts** | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| **Problem-Solving Approach** | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |
| **Accuracy of Solution** | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| **Clarity** | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

| Q. No. / Criteria | Understanding of Problem Context | Application of Concepts | Problem-Solving Approach | Accuracy of Solution | Clarity | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |

| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |